

苏州七狸打卡系统 —— 后端架构与设计答辩文档

本系统后端采用**前后端分离**架构标准，具有轻量、高并发响应、高可扩展性等特点。以下为供答辩使用的后端架构梳理与核心设计讲解。

一、整体架构与技术选型

1. 核心技术栈

- 核心框架:** `FastAPI` (Python 3.10+)
 - 选型依据:** FastAPI 是目前最高效的 Python Web 框架之一，基于 Starlette 和 Pydantic 构建。它天生支持异步 (`asyncio`)，不仅能够承受极高的并发请求，还能自动利用 Type Hints (类型提示) 生成原生的 OpenAPI (Swagger UI) 接口文档，极大地降低了前后端联调成本。
- 关系型数据库:** `SQLite` + `SQLAlchemy` (ORM 框架)
 - 选型依据:** 采用 SQLite 作为数据库底座，不仅部署方便且零配置启动，满足中小型项目的读写需求。配合 SQLAlchemy ORM 技术，实现了将数据库表完全**对象化 (Object Relational Mapping)**，在开发阶段杜绝了繁琐的裸写 SQL，进一步提高了开发安全性 (有效防范 SQL 注入)。
- 对象存储系统:** 阿里云 `OSS`
 - 选型依据:** 系统内置大量用户 UGC 内容 (头像上传、AR 合成照片)。考虑到服务器带宽限制，将重型资源请求全部剥离交由云端 OSS 管理。
- 安全鉴权:** `JWT` (`JSON Web Tokens`) + `passlib`
 - 选型依据:** 贯彻无状态 (Stateless) 准则。服务器不保存用户 Session，用户每次通过客户端携带加密 Token 访问，由服务器实时解析鉴权，大幅节约服务器内存。

二、领域驱动设计 (DDD) 目录分层架构

项目代码严格遵循了高内聚、低耦合的设计哲学：

```
backend_su/
├── app/
│   ├── api/v1/      # (Controller层) 处理路由分发与业务逻辑调用
│   ├── core/        # (Core层) 系统全局配置、安全算法、数据库连接池、云组件挂载
│   ├── models/      # (Model实体层) SQLAlchemy 定义的数据库底座表结构
│   ├── schemas/     # (DTO传输层) Pydantic 数据模型，负责进出站数据的自动清洗与验证
│   └── main.py       # 全局 Application 入口，核心中间件、静态路由的挂载点
```

分层优势阐述：

- 单一职责原则:** `api/` 只管收发 HTTP、`models/` 只负责与数据库打交道、`schemas/` 充当海关负责检查数据合法性。
- 数据双向隔离:** 使用 Pydantic schemas 作为防腐层。前端传来的弱类型数据会自动被转化为强类型结构，且不会将敏感字段 (如查询得到的加密哈希密码) 通过 response 模型直接泄露给前端。

三、核心业务模块拆解与亮点

1. 依赖注入驱动的鉴权体系 (Authentication & DI)

FastAPI 的最大特色。在 `api/v1/auth.py` 中实现了 OAuth2 密码流验证：

- 采用 `bcrypt` 对用户明文密码进行单向哈希加盐加密入库。
- 将验证 Token 生成 `get_current_user_id` 的动作注册为 FastAPI 的 **Dependency Injection (依赖注入)**。
- **代码表现**：对于任何需要登录的接口，只需在路由参数声明 `user_id = Depends(get_current_user_id)`，框架便会在路由执行前瞬间完成拦截、请求头抓取、JWT 解密等数十项验证逻辑。

2. 空间地理位置计算 (GPS Check-in Algorithm)

为了判断用户是否真实“到达”了七狸雕像所处位置：

- 接口接收由移动端 HTML5 Geolocation 获取的高精度原生经纬度。
- 后端利用**半正矢公式 (Haversine Formula)** 在地球球面上对用户坐标 (`lat1, lon1`) 与目标数据库雕像坐标 (`lat2, lon2`) 测量出绝对物理球面距离。
- 检测距离是否处于雕像打卡范围 `radius` (如 500 米内)，成功后写入多维打卡统计状态 `checkins/statistics`，并触发进度里程碑 (如“姑苏漫步者”荣誉的解锁)。

3. UGC 存储的高可用容灾方案 (Storage Fallback)

为了防止答辩或线上网络波动导致阿里云 OSS 失联：

- 在 `upload.py` 和 `photos.py` 的重逻辑中，撰写了“尝试云上传，失败则本地写盘”的 **Fallback 回退机制**。
- `main.py` 开启了虚拟静态挂载路由配合前置 Nginx 的反向代理，无论照片是去了云端的 URL，或者是留在了本地 `/api/v1/uploads/...` 路径，前端一律能做到无感正确渲染。

4. 前后端分离部署的网关策略 (API Gateway)

本系统前端托管于单页应用 (SPA) 架构中，必定存在 **CORS 跨域资源漏洞** 风险：

- 在 FastAPI 侧不仅严格配置了允许源 (`allow_origins` 等安全头)，防止 CSRF 窃取。
- 在宝塔面板 (Nginx) 中执行了正统的 API 代理请求：前端访问 `/api` 会被无缝转发至本地服务器对应的进程端口。此举使得我们在开启 HTTPS (443端口) 后，顺利绕过了现代浏览器对于“混合内容 (HTTPS 调 HTTP)”的安全封锁策略。

四、总结可扩展性分析

该后端系统尽管目前应用于打卡项目，但它具备极强的发展潜力：未来完全可以通过迁移 `database.py` 中的引擎源至 MySQL/PostgreSQL 直接演化为分布式支撑架构；在遭遇流量波峰时也能利用 FastAPI 内建的高性能 `async/await` 非阻塞处理机制平切高并发场景。是现代化技术栈中极具工程参考价值的一套设计方案。